What Is AI, Economically?

(And What Does It Look Like Inside?)

Prof. Eduardo Zambrano

ECON 470 — Microeconomics of Artificial Intelligence

Welcome to ECON 470

Microeconomics of Artificial Intelligence

. . .

> “We use microeconomic theory to understand artificial intelligence — not as a buzzword, but as an economic phenomenon with agents, incentives, markets, and policy consequences.”

Course Logistics

- **Textbook:** Gans, *The Microeconomics of Artificial Intelligence* (MIT Press, 2025) — free PDF via MIT Press Direct Open Access
- **Grading:**
  - Homework: 30% (5 problem sets)
  - Presentations: 20% (Digitalist Papers essays)
  - Midterm: 20%
  - Final: 30%
- **Late work:** 20% penalty per calendar day

The Presentation Assignment

Each student presents a **Digitalist Papers** essay and leads a class discussion.

- Essays from Stanford Digital Economy Lab, *Volume 2: The Economics of Transformative AI*
- Authors include Bengio, Autor, Stiglitz, Athey, Agrawal & Gans
- Presentations begin **Week 4** (~15 min presentation + ~10 min discussion)
- Start browsing now: digitalistpapers.com/volume2
- Sign-up sheet goes out next week

Now let’s talk about what we’re actually here to study.

The Big Question

What is AI, economically?

AI Is Not What You Think

AI is **not** “artificial intelligence” in the sci-fi sense.

Economically, AI is a technology that dramatically **reduces the cost of prediction**.

— Agrawal, Gans & Goldfarb (2018); Gans (2025)

The Anatomy of a Decision



AI has made **prediction** dramatically cheaper.

When prediction gets cheap, it reshapes every decision where prediction is an input.

Examples of Prediction

Decision	Prediction	Judgment
What to wear	Will it rain?	How much do I mind getting wet?
Route to work	Traffic conditions	Am I willing to be late?
Spam filter	Is this email spam?	Cost of missing a real email vs. reading spam?
Medical diagnosis	Disease probability	Treatment risks vs. benefits?

The **prediction** is what AI does. The **judgment** is what humans supply.

“So that’s the economic framing. AI is a prediction machine.

But what does a prediction machine actually look like?

Let me show you one — the entire thing — in 200 lines of code.”

Inside a GPT

Andrej Karpathy's `microgpt.py`

What Is This Thing?

A **complete, working GPT** — the same architecture behind ChatGPT — in 200 lines of pure Python.

- No libraries. No dependencies. Just `os`, `math`, `random`.
- Trains on 32,000 baby names
- Learns to generate new plausible names
- 4,192 parameters (ChatGPT: hundreds of billions)

> “This is to ChatGPT what a paper airplane is to a 747. Same physics. Different scale.”

Introducing `microgpt.py`

```python
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

Before We Dive Into the Code…

Let’s build some visual intuition for what’s happening inside a transformer.

3Blue1Brown: *But what is a GPT?*

~27 min — embeddings, attention, and softmax, visually explained

The Dataset

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

32,000 baby names. That’s the training data — the raw material the model will learn from.

The Tokenizer

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

Economic connection

Every AI system starts here — converting messy real-world information into structured data. This conversion has costs and makes choices. What you include and exclude shapes everything downstream.

The Autograd Engine

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

The `Value` class tracks every computation so the model can learn from its mistakes. Each operation records how to “undo” itself — that’s the chain rule from calculus, implemented in 44 lines.

Parameter Initialization

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

Where Do 4,192 Parameters Come From?

With `vocab_size = 27`, `n_embd = 16`, `block_size = 16`, `n_head = 4`:

Matrix	Dimensions	Count
Token embeddings (`wte`)	27×16	432
Position embeddings (`wpe`)	16×16	256
Attention: Q, K, V, O	$4 \times (16 \times 16)$	1,024
MLP: fc1	64×16	1,024
MLP: fc2	16×64	1,024
Output head (`lm_head`)	27×16	432

Total: $432 + 256 + 1{,}024 + 1{,}024 + 1{,}024 + 432 = 4{,}192$

Economic connection

These parameters are the model's **capital stock**. Training is the investment process. Who owns these 4,192 numbers, who can copy them, who can inspect them — that's intellectual property (Week 9).

Helper Functions

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

Three small functions that the model uses repeatedly:

- `linear`: matrix multiplication — the basic building block
- `softmax`: turns raw scores into a **probability distribution**
- `rmsnorm`: normalizes values to keep them stable

Before the Forward Pass…

The forward pass is where the **attention mechanism** lives — the core innovation behind modern AI. Let's build visual intuition first.

3Blue1Brown: *Attention in Transformers, Visually Explained*

~26 min — how tokens "talk to each other" through queries, keys, and values

The GPT Forward Pass

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

This is the core: given a character and its position, predict the next character.

1. Look up **embeddings** (lines 109-112)
2. Run **attention** — “what should I pay attention to?” (lines 114-134)
3. Run the **MLP** — combine and refine (lines 135-141)
4. Output **logits** — scores for each possible next character (line 143)

Think of the 4 attention heads as **4 analysts** in a room, each looking at the same data but attending to different features.

Optimizer Setup

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

Adam: a refined version of gradient descent that adapts learning rates per parameter. Three lines to set it up.

The Training Loop

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

Repeat 1,000 times: pick a name, predict each character, measure the error, adjust all parameters.

- **Line 167:** `loss_t = -probs[target_id].log()` — cross-entropy loss. This is the **objective function**.
- **Line 172:** `loss.backward()` — backpropagation. How did each parameter contribute to the error?
- **Lines 176-182:** Adam update. Nudge each parameter to reduce the loss.

Loss starts at **~3.3** (random guessing among 27 characters: $-\ln(1/27) \approx 3.3$) and drops to **~2.4** — the model has learned something.

Economic connection

The loss function determines what "good prediction" means. Choosing it is an act of **judgment**, not prediction. The model minimizes prediction error, just as a firm minimizes cost.

> "Training is like tuning a radio with 4,192 knobs. You nudge each knob a tiny bit in the direction that makes the static quieter. After 1,000 rounds, you can hear the music."

Inference

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

We have reached line 200. After training, the model generates new names one character at a time.

- **Temperature** (line 187) controls randomness: low = safe/common, high = creative/risky
- The model predicts, samples, predicts, samples… until it produces a BOS (end) token

That’s a GPT. The whole thing. 200 lines.

Every GPT you’ve ever used — ChatGPT, Gemini, Claude — is this same architecture, scaled up by a factor of a million.

Now when we talk about the **economics of AI prediction**, you know exactly what “prediction” means.

Measuring What the Model Learned

The loss went from **~3.3** to **~2.37**.

We know $3.3 \approx -\ln(1/27)$ — random guessing among 27 characters.

Question: If 3.3 means “27 equally likely choices,” what does 2.37 mean?

Answer: Loss $= -\ln(1/k)$ implies $k = e^{\text{loss}}$. So $e^{2.37} \approx 10.7$.

The model narrowed its uncertainty from **27 equally likely characters** down to roughly **10–11 effective choices** per position.

That’s what “learning” means quantitatively: **reducing uncertainty**, measured in nats. This is exactly what Gans means by “AI reduces the cost of prediction.”

“Everything Else Is Just Efficiency”

	`microgpt.py`	**Production LLM**
Tokens	27 characters	~100,000 subwords (BPE)
Parameters	4,192	hundreds of billions
Data	32,000 baby names	trillions of tokens
Compute	your laptop, ~1 min	thousands of GPUs, months
Code	200 lines of Python	millions of lines + infrastructure

The **algorithm is identical**. The differences are all engineering and scale.

Wrap-up

For Next Class

Read: Gans, Chapter 2 (Advances in Machine Learning)

We'll connect the history in Chapter 2 to what you just saw, and then we'll **break the model** to see what prediction *can't* do.

Browse the microGPT code on your own.

gist.github.com/karpathy/8627fe009c40f57531cb18360106ce95

Don't worry about understanding every line — just try to get a feel for what each block of code does at a high level. Each pass, more will make sense.